
Dependence Testing

Allen and Kennedy, Chapter 3 thru Section 3.3.2

Applying Single Subscript Tests

- ZIV Test
- SIV Test
 - Strong SIV Test
 - Weak SIV Test
 - Weak-zero SIV
 - Weak Crossing SIV
- SIV Tests in Complex Iteration Spaces

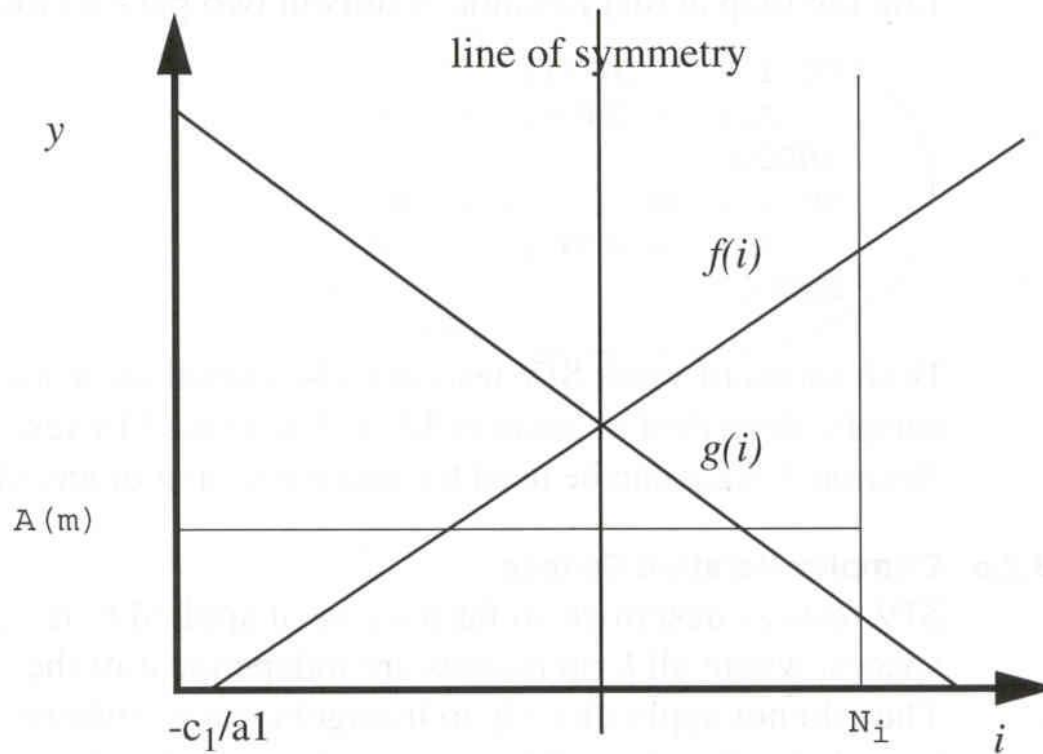
Weak-crossing SIV Test

- Special case of Weak SIV where the coefficients of the index are equal in magnitude but opposite in sign
- The test consists merely of checking whether the solution index is within loop bounds

$$i = \frac{c_2 - c_1}{2a_1}$$

Weak-crossing SIV Test

Geometric View of Weak-crossing SIV Subscripts



Weak-crossing SIV & Loop Splitting

```
DO i = 1, N  
S1      A(i) = A(N-i+1) + C  
ENDDO
```

- Can we extract parallelism from the loop?

Weak-crossing SIV & Loop Splitting

```
DO i = 1, N
S1      A(i) = A(N-i+1) + C
ENDDO
```

This loop can be split into...

```
DO i = 1, (N+1)/2
      A(i) = A(N-i+1) + C
ENDDO

DO i = (N+1)/2 + 1, N
      A(i) = A(N-i+1) + C
ENDDO
```

The General Case

- We must relax our restrictions on f and g to let them be arbitrary functions.
- A dependency exists if

$$h(x_1, x_2, \dots, x_n, y_1, y_2, \dots, y_n) = f(x_1, x_2, \dots, x_n, y_1, y_2, \dots, y_n) - g(x_1, x_2, \dots, x_n, y_1, y_2, \dots, y_n) = 0$$

- has an integral solution
- The above equation is known as a Diophantine equation.

Linear Diophantine Equations

- For simplicity, assume that

$$f(x) = a_0 + a_1x_1 + \dots + a_nx_n$$

$$g(x) = b_0 + b_1y_1 + \dots + b_ny_n$$

- Then, we're looking for a solution of

$$h(x) = a_0 - b_0 + a_1x_1 - b_1y_1 + \dots + a_nx_n - b_ny_n = 0$$

- Rearranging terms, we get the linear Diophantine Equation:

$$a_1x_1 - b_1y_1 + \dots + a_nx_n - b_ny_n = b_0 - a_0$$

Exercise

```
for I = 1 to 9 do {  
  S1: A(2*I) = ...  
  S2: ... = A(4*I+1)  
}
```

- Can the dependency exist in the loop?

```
for I = 1 to 100 do {  
  S1: A(5*I) = ...  
  S2: ... = A(10*I+2)  
}
```

```
for I = 1 to 100 do {  
  S1: A(5*I) = ...  
  S2: ... = A(10*I+5)  
}
```

- Can the dependency exist in the loop?

Can the dependency exist in the loop?

GCD Test

- As a result, the following dependence equation has a solution iff $\gcd(a_1, \dots, a_n, b_1, \dots, b_n)$ divides $b_0 - a_0$

$$a_1x_1 - b_1y_1 + \dots + a_nx_n - b_ny_n = b_0 - a_0$$

Example

```
for I = 1 to 10 do {  
  S1: A(I) = ...  
  S2: ... = A(I)  
}
```

- The dependence equation is: $I1 - I2 = 0$, with,
- $A0 = 0$ $A1 = 1$, $B0 = 0$, $B1 = 1$
- $GCD(A1, B1)|(B0 - A0) = GCD(1,1)|0 = 1|0$, which is true; hence dependence exists
- However, the direction vector, the direction of dependence, (S1 to S2 or vice-versa), and the type of dependence (flow, anti-, or output) are not indicated by the GCD test

Exercise-1

```
for I = 1 to 9 do {  
  S1: A(I) = ...  
  S2: ... = A(10-I)  
}
```

- Use GCD test to detect if dependency is present?

Exercise-2

```
for I = 1 to 10 do {  
  S1: A(2*I) = ...  
  S2: ... = A(4*I+3)  
}
```

- Use GCD test to detect if dependency is present?

Exercise-3

```
for I = 1 to 10 do {  
  S1: A(I) = ...  
  S2: ... = A(I+10)  
}
```

- Use *GCD* test to detect if dependency is present?

GCD Test - Summary

- The GCD test is quick but not very effective in practice
- The GCD test indicates dependence whenever the dependence equation has a solution anywhere, not necessarily within the region imposed by the loop bounds

Banerjee Test

- Equation is:

$$a_1x_1 - b_1y_1 + \dots + a_nx_n - b_ny_n = b_0 - a_0$$

- Compute the Lower Bound (LB) and Upper Bound (UB) of the left hand side by considering all the loop bounds.

$$LB \leq a_1x_1 - b_1y_1 + \dots + a_nx_n - b_ny_n \leq UB$$

- If the above condition is satisfied then dependency exists

Example

```
for I = 1 to 10 do {  
  S1: A(I) = ...  
  S2: ... = A(I)  
}
```

Example

```
for I = 1 to 10 do {  
  S1: A(I) = ...  
  S2: ... = A(I)  
}
```

- The dependence equation is: $I1 - I2 = 0$
- LB of $I1 - I2$?
- UB of $I1 - I2$?

$$-9 < I1 - I2 < 9$$

- Dependency exists. $-9 < 0 < 9$ is

Exercise-1

```
for I = 1 to 9 do {  
  S1: A(I) = ...  
  S2: ... = A(10-I)  
}
```

- Use banerjee test to detect if dependency is present?

Exercise-2

```
for I = 1 to 10 do {  
  S1: A(2*I) = ...  
  S2: ... = A(4*I+3)  
}
```

- Use banerjee test to detect if dependency is present?

Exercise-3

```
for I = 1 to 10 do {  
  S1: A(I) = ...  
  S2: ... = A(I+10)  
}
```

- Use banerjee test to detect if dependency is present?

Exercise-4

```
for ( int i =0; i <=10; i ++ ) {  
    for ( int j =0; j <20; j ++ ) {  
        for ( int k =0; k <30; k ++ ) {  
            a [ i *15+ j *10+ k *9] = c ;  
            b = a [ i *5+ j *8+ k  
                *4+300];  
        }  
    }  
}
```

- Use banerjee test and GCD test to detect if dependency is present?

Exercise: Parallelization

```
for(int i=0; i<10; i++) {  
  S1:    A[i]=B[i]+10;  
        for(int j=0; j<20; j++) {  
          for(k=0; k<30; k++) {  
S2:      C[i][j][k]=A[i-10]+E[i][j-5][k];  
S3:      D[i][j]=A[i-1]+C[i-1][j-2][k]+D[i-1][j];  
S4:      E[i][j][k]=D[i][j-4]+C[i][j][k];  
          }  
        }  
  }  
}
```

- Construct the dependency matrix/table
- Construct dependency graph
- Extract parallelism

Questions?